

Lesson 3: CSS Basics — Colors, Fonts, and Spacing

Mission Brief

Your ship's blueprint is solid, Navigator — now it's time to paint the hull and light up the control panels. CSS (Cascading Style Sheets) is how you take bare HTML structure and give it color, typography, and breathing room. Today you'll connect your first external stylesheet and transform a plain page into something that looks intentional.

Learning Objectives

- Link an external CSS file to an HTML document correctly
- Write CSS rules using selectors, properties, and values
- Apply colors using hex, `rgb()`, and named values
- Set font family, size, weight, and line-height
- Control spacing using margin, padding, and the box model

What You'll Build

A styled version of your Lesson 2 "Crew Profile" page, with a real color palette, custom fonts, and consistent spacing — plus a standalone `styles.css` file.

Prerequisites

Lesson 2: a semantically structured HTML page with headers, sections, and lists to style.

Key Concepts

- Selector, property, value, declaration, rule
- External vs. inline vs. internal CSS
- Color formats: hex, `rgb()`, named colors
- `font-family`, `font-size`, `font-weight`, `line-height`
- The CSS Box Model: content, padding, border, margin
- `class` and `id` selectors

Concept Explanation

CSS works by writing **rules**: a **selector** (which HTML elements to target) followed by a block of **declarations** in curly braces, each declaration being a **property: value;** pair. For example, `p { color: navy; }` selects every `<p>` element and sets its text color to navy. There are three ways to attach CSS to HTML: **inline** (a `style="..."` attribute directly on a tag — avoid this, it doesn't scale), **internal** (a `<style>` block inside `<head>` — fine for quick tests), and **external** (a separate `.css` file linked via `<link rel="stylesheet" href="styles.css">` in the `<head>` — the professional standard, because it keeps content and presentation separate and lets one file style many pages).

Selectors can target elements directly (`p` , `h1`), or by **class** (`.card` targets any element with `class="card"`) or **id** (`#main-nav` targets the one element with `id="main-nav"` — ids must be unique per page). Classes are reusable across many elements; ids are meant for a single unique element, often used as anchor targets or JavaScript hooks.

Colors in CSS can be written as **named colors** (`red` , `steelblue`), **hex codes** (`#1a2b3c` , where each pair of characters is red/green/blue intensity from `00` to `ff`), or functional notation like `rgb(26, 43, 60)` or `rgba(26, 43, 60, 0.8)` (the fourth value adds transparency, called alpha). Professional designs pick a small, deliberate **palette** — usually one or two primary colors, a neutral background/text pair, and an accent color — rather than random colors per element.

Typography is controlled with `font-family` (a stack of fallback fonts, e.g. `font-family: "Poppins", Arial, sans-serif;` — the browser tries each in order until one is available), `font-size` (commonly in `px` , `rem` , or `%`), `font-weight` (numeric 100-900, or keywords like `bold`), and `line-height` (the vertical space a line of text occupies, usually a unitless multiplier like `1.5` for comfortable reading).

Finally, every HTML element is rendered as a rectangular box, described by the **box model**: at the center is the **content**, surrounded by **padding** (space between content and the border), then the **border** itself, then **margin** (space outside the border, separating this box from its neighbors). Understanding that "spacing" is really "padding vs. margin" is one of the most important mental models in all of CSS — padding pushes content inward, margin pushes other elements away.

Guided Coding

```

/* styles.css */

/* Reset a few defaults so all browsers start from the same baseline */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* padding/border are now INCLUDED in width, not added on top */
}

body {
  font-family: "Segoe UI", Arial, sans-serif;
  line-height: 1.6;
  color: #1a1a2e; /* deep space navy text */
  background-color: #f4f6fb; /* soft off-white background */
}

header {
  background-color: #1a1a2e;
  color: #ffffff;
  padding: 24px 16px; /* 24px top/bottom, 16px left/right */
}

header h1 {
  font-size: 1.8rem;
  font-weight: 700;
}

nav ul {
  list-style: none; /* remove bullet points */
  display: flex; /* lay list items out in a row (previewed here, covered fully next lesson) */
  /*
  gap: 16px;
  margin-top: 12px;
  */
}

nav a {
  color: #a6c1ff;
  text-decoration: none;
  font-weight: 600;
}

nav a:hover {
  text-decoration: underline;
}

main {
  max-width: 800px;
  margin: 32px auto; /* auto left/right margin centers the block */
}

```

```
padding: 0 16px;
}

section {
  background-color: #ffffff;
  border: 1px solid #dde3f0;
  border-radius: 8px;
  padding: 24px;
  margin-bottom: 24px;
}

section h2 {
  color: #4361ee;
  font-size: 1.4rem;
  margin-bottom: 12px;
}

footer {
  text-align: center;
  padding: 16px;
  color: #6b7280;
  font-size: 0.9rem;
}
```

```
<!-- add this single line inside <head>, after <title> -->
<link rel="stylesheet" href="styles.css" />
```

Code Walkthrough

- `* { margin: 0; padding: 0; box-sizing: border-box; }` — a universal reset. `box-sizing: border-box` is important: it means when you later set `width: 300px`, padding and border are included inside that 300px instead of adding to it, which makes layout math far more predictable.
- `body { font-family, line-height, color, background-color }` — sets the page-wide default typography and colors; child elements inherit these unless overridden.
- `header { background-color, color, padding }` — colors the header band and pads its content so text isn't jammed against the edges; `padding: 24px 16px` is shorthand for top/bottom then left/right.
- `nav ul { list-style: none; display: flex; gap: 16px; }` — removes bullets and arranges nav items horizontally with even spacing (a first taste of Flexbox, covered fully in Lesson 4).
- `nav a:hover` — a **pseudo-class** selector; styles only apply while the mouse hovers over a link.
- `main { max-width: 800px; margin: 32px auto; }` — caps the content width for readability and centers the block horizontally using `auto` margins.
- `section { border-radius: 8px; }` — rounds the corners of each card-like section.

Student Mission

1. Create `styles.css` with the rules above and link it from your Lesson 2 HTML page.
2. Change the color palette to three colors of your own choosing (one dark, one light, one accent) and apply them consistently.
3. Add a `.highlight` class in your CSS that sets a distinct `background-color` and apply it to one paragraph in your HTML using `class="highlight"`.
4. Adjust `line-height` and `font-size` on your body text until it feels comfortable to read.

Challenge

Add a `rgba()` background color with partial transparency to your `<aside>` element, and add a `border-left: 4px solid <color>;` accent stripe to it. Then create a second `.css` "theme" file with a different palette and manually swap which one is linked to see how completely CSS can restyle identical HTML.

Experiment/Extension

Change `box-sizing: border-box` to `content-box` (or delete the rule) on an element with both `width` and `padding` set, and observe how its total rendered size changes. Note what you observe in a CSS comment.

Common Mistakes

- Forgetting the `<link>` tag, so the CSS file never loads (checked instantly by DevTools' Network tab — a 404 there means a wrong path).
- Confusing padding and margin — padding is inside the border, margin is outside.
- Using an id selector when a class was meant (ids can only be used once per page).
- Forgetting units on non-zero values (`margin: 10;` is invalid — must be `10px` , `10rem` , etc.).
- Overly specific or duplicated selectors making styles hard to override later.

Debugging Tips

- Open DevTools → Elements → click an element → check the **Styles** pane on the right; it shows every CSS rule applying to that element and which ones are being overridden (shown with strikethrough).
- The **Computed** tab in DevTools shows the final box model dimensions (content/padding/border/margin) as a visual diagram — invaluable for spacing bugs.
- If your CSS file isn't applying at all, check the Network tab for a failed (red) request to `styles.css` — that usually means a typo in the file path.

Check Your Understanding

1. What are the three ways to attach CSS to an HTML page, and which is preferred for real projects?
2. What is the difference between a class and an id selector?
3. In the box model, what is the difference between padding and margin?
4. What does `box-sizing: border-box` change about how width is calculated?
5. Name two ways to express color in CSS.

Mini Quiz + Answer Key

Quiz 1. Which HTML tag links an external stylesheet? a) `<style>` b) `<script>` c) `<link>` d) `<css>` 2. True or False: An id can be reused on multiple elements in the same page. 3. What property controls the space between a line of text and the next line? 4. Which CSS box-model layer sits directly outside the border? 5. What selector targets `class="card"` ?

Answer Key 1. c) `<link>` 2. False — ids must be unique per page. 3. `line-height` 4. margin 5. `.card`

Lesson Recap

- External CSS (via `<link>`) is the professional way to style HTML.
- Selectors (element, class, id) target which elements a rule applies to.
- Colors can be named, hex, or `rgb()/rgba()`.
- Typography is shaped by font-family, font-size, font-weight, and line-height.
- The box model (content → padding → border → margin) governs all spacing.

Homework

Style your Lesson 1 `mission-log.html` from scratch with its own new `styles.css` : pick a 3-color palette, set a font stack, and use padding/margin so nothing touches the edge of the browser window.

Portfolio Project Connection

This lesson's color palette, typography choices, and box-model spacing rules become the visual foundation of `styles.css` in **all three projects** — especially the polished, cohesive look required in **Project 01 (Personal landing page)**.